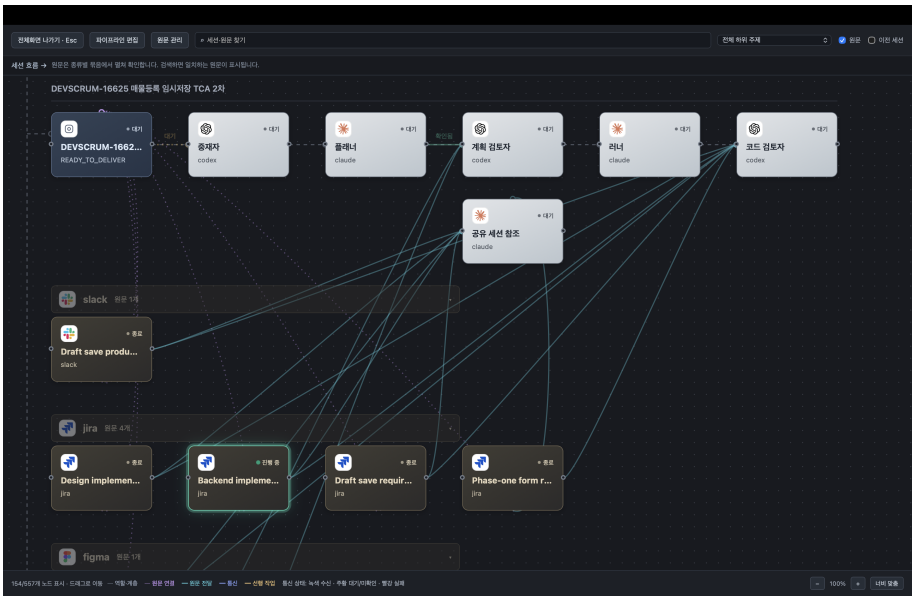
Claude/Codex 공동 개발 작업 자동화

같은 대화에서 계획/구현/검토를 이어 가고, 실행과 결과를 기록하는 도구.



이 프로젝트를 시작한 계기

한 에이전트에 계획과 구현을 맡겼을 때 필요한 작업이 빠지거나, 잘못된 전제로 진행이 막히고 일부 현상을 과하게 해석하는 일을 겪었다. Claude와 Codex의 의견을 직접 주고받아 계획을 다듬자, 서로 놓친 문제를 보완해 합의한 계획의 완성도가 높아졌다.

이 방식을 계속 활용하고 싶었지만, 매번 응답을 전달하고 합의를 진행하기는 어려웠다. 그래서 계획 합의 → 구현 → 교차 검토 → 검증을 자동으로 이어 가는 Consensus Room을 만들었다. 작업 범위와 우선순위, 설계 판단과 계획 승인은 직접 맡는다.

시스템 개요

큰 변경의 전략과 순서를 합의하는 플랜모드와, 요구사항이 자주 바뀌는 업무를 처리하는 티켓모드를 둔다. 외부 자료의 변경은 자동으로 수집해 작업 세션에 전달하고, 엔진은 실행 조건과 결과 기록을 맡는다.

실제 업무에 적용한 결과

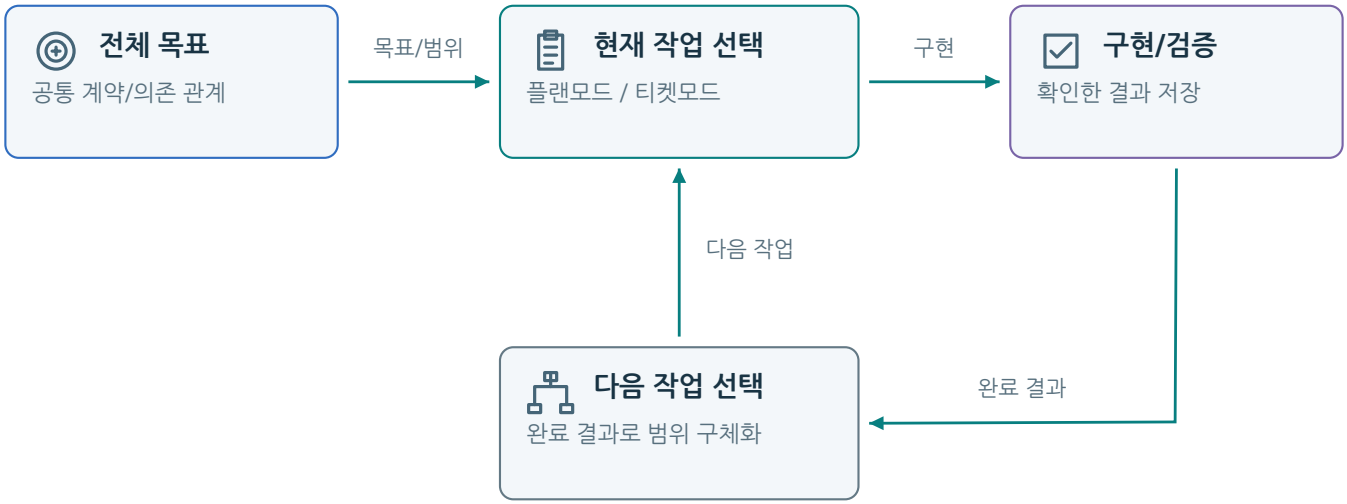
10월 개편 전 엔진으로 운영 iOS 앱의 Swift 6 전환을 약 4주에 완료했다. 공통 모듈부터 앱과 테스트까지 단계별로 검증해 9월 23일 개발 브랜치에 통합했고, CI 검사로 Swift 5 설정의 재유입도 막았다.

세션 그래프: 역할별 참여자와 외부 원문 연결. 화면의 배정과 상태는 촬영 당시 기준.

플랜모드와 티켓모드의 선택 기준

변경의 성격에 따라 계획 합의부터 시작하거나, 구현과 검토를 바로 이어 간다.

초기에는 모든 작업을 계획부터 시작해, 요구사항이 바뀔 때마다 계획 검토를 되풀이했다. 큰 변경은 전략과 순서를 먼저 합의하고, 스프린트 업무는 변경된 자료와 우선순위에 맞춰 바로 구현할 수 있도록 두 모드로 나눴다.



1. 플랜모드: 대규모 변경의 전략과 순서 합의

Swift 6 전환처럼 여러 모듈을 함께 바꾸는 작업에 사용한다. 전체 목표와 단계별 의존 관계를 정하고, Claude와 Codex가 검토한 계획을 내가 승인하면 구현 → 교차 검토 → 검증으로 이어진다. 현재 계획과 승인한 판이 같은지도 확인한다.

계획은 여러 Markdown 파일로 나누되, 플래너와 리뷰어는 같은 대화를 이어 쓴다. 엔진은 계획의 스냅샷과 변경분, 승인한 판을 보존한다.

2. 티켓모드: 바뀐 자료와 우선순위에 맞춘 구현

요구사항과 디자인, API가 자주 바뀌는 스프린트 업무에 사용한다. Jira/Slack/Figma/백엔드 문서의 변경을 자동 수집해 작업 세션에 전달하고, 내가 우선순위와 반영 범위를 정한다. 이후 구현 → 교차 검토 → 변경 범위에 맞는 검증으로 이어 간다.

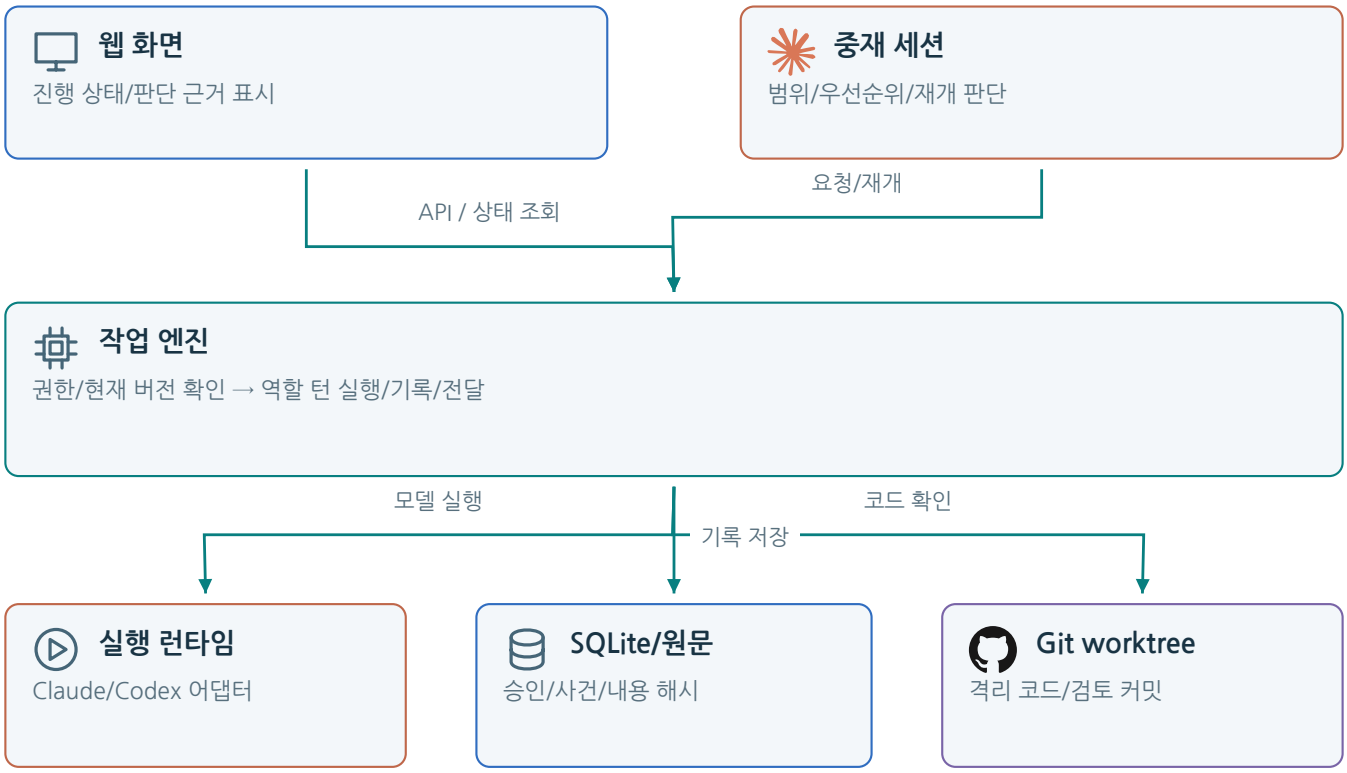
두 모델의 계획 합의를 필수로 두지 않아 변경마다 전체 계획을 다시 합의하지 않는다. 구현 뒤 코드 리뷰와 검증은 유지한다. 내부에서는 플랜모드를 계획 작업(planned), 티켓모드를 일반 작업(ticket)으로 구분한다.

src/server/planBundle.ts / src/server/engine/planning.ts / src/shared/prompts.ts

화면/엔진/실행 도구의 역할

작업자가 내용을 판단하고, 엔진은 실행 조건/중복 실행/결과 보존을 책임진다.

React/Vite는 상태를 보여 주고, Node.js/TypeScript/Fastify 서버는 허용된 행동을 실행한다. SQLite는 승인/배정/사건/중단 위치를 보존한다. 엔진이 지적의 의미까지 여러 곳에서 판정하던 구조를 줄이고, 역할별 응답을 다음 세션에 전달하도록 책임을 좁혔다.



역할별 구현 책임

React/Vite는 화면을, Node.js/TypeScript/Fastify는 서버를 구성한다. 공통 실행 런타임은 Claude/Codex CLI의 옵션과 출력 차이를 어댑터 뒤로 모은다. Git 작업 트리는 코드 변경을 격리하며, 원문과 결과 파일은 내용 해시로 식별한다.

공통 실행과 정책의 경계

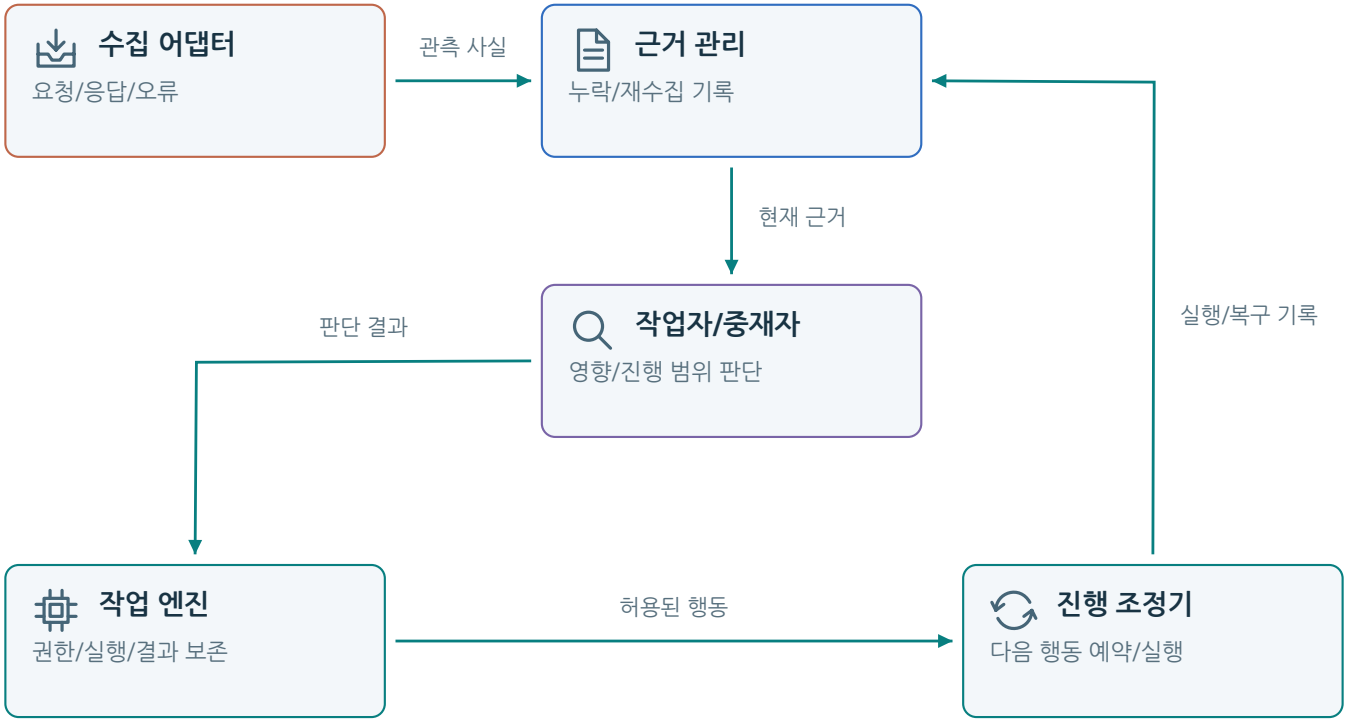
엔진과 호스트 리뷰는 같은 실행 계약을 쓰지만 각자의 승인과 예산을 소유한다. 런타임이 운영 DB를 대신 수정하거나 리뷰 승인을 만들어 내지 않는다. 이렇게 나누면 공급자 실행 방식을 고쳐도 작업 승인 규칙까지 바꾸지 않아도 된다.

src/server/runtime/ / src/server/engine/ / src/server/database.ts

계층별 책임과 공통 계약

수집 사실과 작업 판단을 분리하고, 다음 행동은 엔진의 공통 조건으로 실행.

같은 원문 변경을 수집기, 계획 게이트, 구현 게이트가 각각 해석했다. 한 곳의 정지를 풀어도 다른 곳이 다시 막았다. 복구 코드를 덧붙일수록 어느 계층의 판단이 정본인지 추적해야 하는 경로도 늘었다.



수집부터 실행까지의 책임

수집기는 원문/변경/조회 실패를 기록한다. 작업자와 중재자는 그 사실로 진행 범위를 판단한다. 엔진은 현재 실행의 소유권, 승인 조건, 결과 형식을 검사하고 다음 역할에 응답을 보낸다. 원문 변경만으로 계획과 세션을 초기화하지 않는다.

공통 처리로 모으고 옛 판단 경로를 삭제

같은 원인을 여러 곳에서 다르게 분류하던 쟁점 판정, 확인 턴, 재작성 원장과 옛 계획 루프를 제거했다. 실행 호출과 결과 보존은 공통 경계로 모았다. 다만 권한, Git HEAD, 실행 신원처럼 서로 다른 위험을 막는 검사는 유지했다.

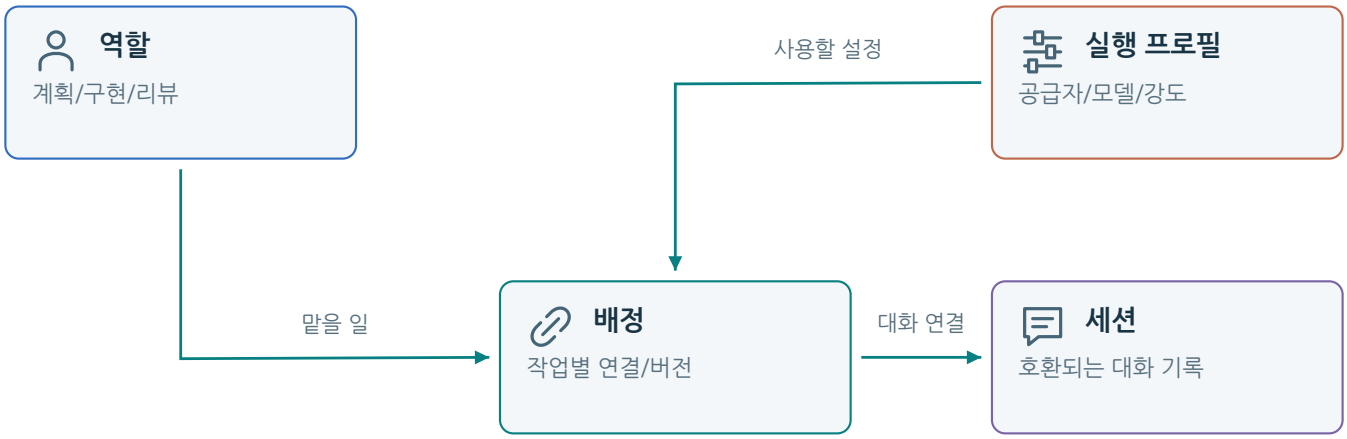
삭제 순서도 어려웠다. 옛 소비처가 남은 상태에서 복구 코드를 먼저 지우면 재개가 막혔다. 새 흐름을 먼저 공급하고 호출부를 바꾼 뒤, 남은 소비처와 검사를 함께 정리했다. 이름 검색만으로 간접 소비를 놓친 사례도 있었다.

src/server/engine/core.ts / src/server/engine/turnExecutor.ts / src/server/runtime/invoke.ts

역할/프로필/배정/세션

역할은 책임, 프로필은 실행 설정, 배정은 연결 관계, 세션은 대화 기록.

설계자/구현자/검토자라는 역할은 허용할 행동과 결과 형식을 설명한다. 실행 프로필은 공급자/모델/추론 강도/공급자별 옵션을 담는다. 배정은 어느 작업에서 어떤 프로필을 쓰는지와 그 버전을 연결한다. 세션은 해당 배정의 대화 기록이다.



배정 변경과 호환성

이 구분이 없으면 중재자를 Claude에서 Codex로 바꿀 때 역할과 무관한 설정까지 다시 만들어야 한다. 개편에서는 배정과 기능 지원 여부를 먼저 확인한 뒤 공통 실행 요청으로 변환한다. 전역 배정을 작업별 배정으로 덮어쓸 수 있지만, 기록 없이 설정을 바꾸지는 않는다.

새 공급자의 연결 조건

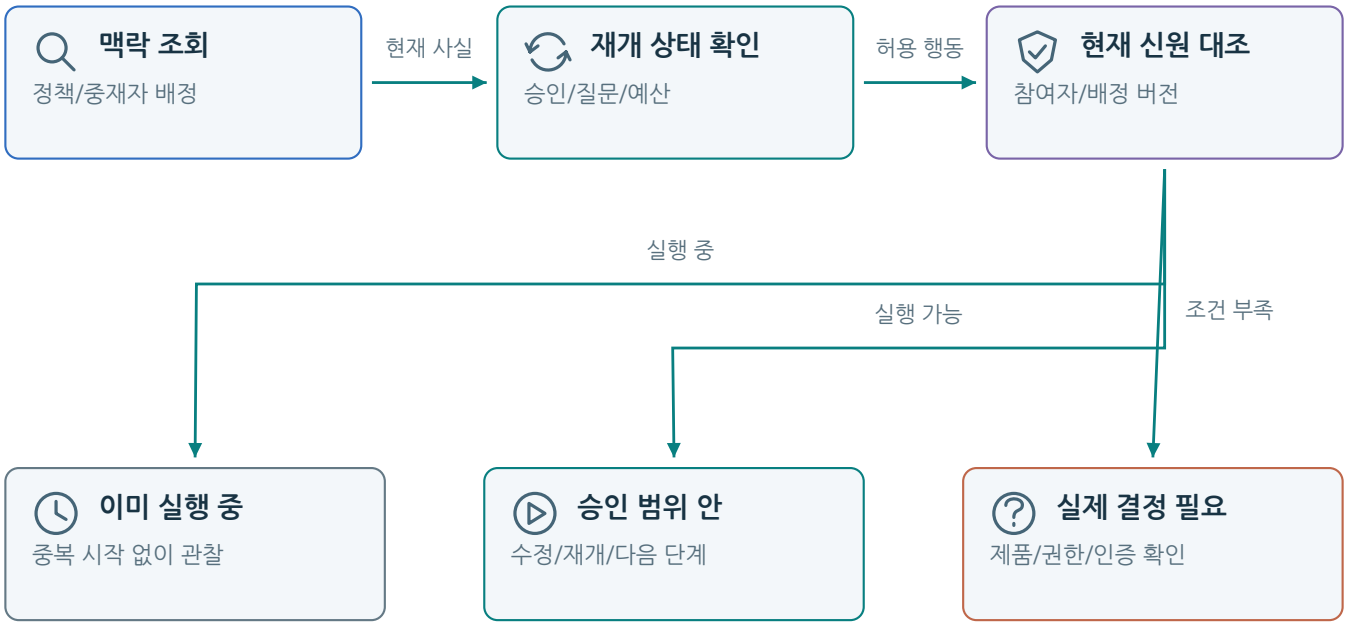
실제 어댑터는 Claude와 Codex 두 개다. 임의의 AI 도구를 연결하면 곧바로 동작하는 구조는 아니다. 새 공급자는 실행/권한/세션/사용량/결과 파싱 계약을 구현해야 한다. ultracode 같은 옵션도 공급자별 계약으로 남겨 공통 추론 강도와 혼동하지 않는다.

src/shared/roles.ts / src/server/roleAssignments.ts / src/server/runtime/providers.ts

중재자의 확인 순서

대화의 마지막 문장보다 서버의 현재 상태/허용 행동/배정 신원을 먼저 확인.

중재자는 어떤 작업이 진행 중인지, 무엇 때문에 멈췄는지, 남은 질문과 예산이 무엇인지 확인한다. 이미 승인된 범위 안의 수정과 다음 준비된 단계를 조율하고, 서버에 저장된 자동 진행 상태를 확인한다. 제품 결정/인증/권한/예산처럼 실제로 부족한 조건은 이유를 남기고 요청한다.



중재자 교체와 상태 확인

대화의 마지막 문장만 보고 재개하지 않도록 현재 맥락 조회, 재개 상태 확인, 역할 배정 신원 확인 순서를 둔다. 이전 승인과 질문을 유지하고 이미 실행 중인 작업을 중복 시작하지 않게 하는 절차다.

하위 작업의 연결 누락과 복구

상위 작업에는 중재 세션이 연결돼 있었지만 새 하위 작업은 미배정으로 표시됐다. 배정 조회가 자기 작업과 전역 설정만 확인해 상위 연결을 놓친 것이 원인이었다. 가장 가까운 상위 중재자 배정을 조회하되 원래 버전과 세션을 유지하도록 고쳤다.

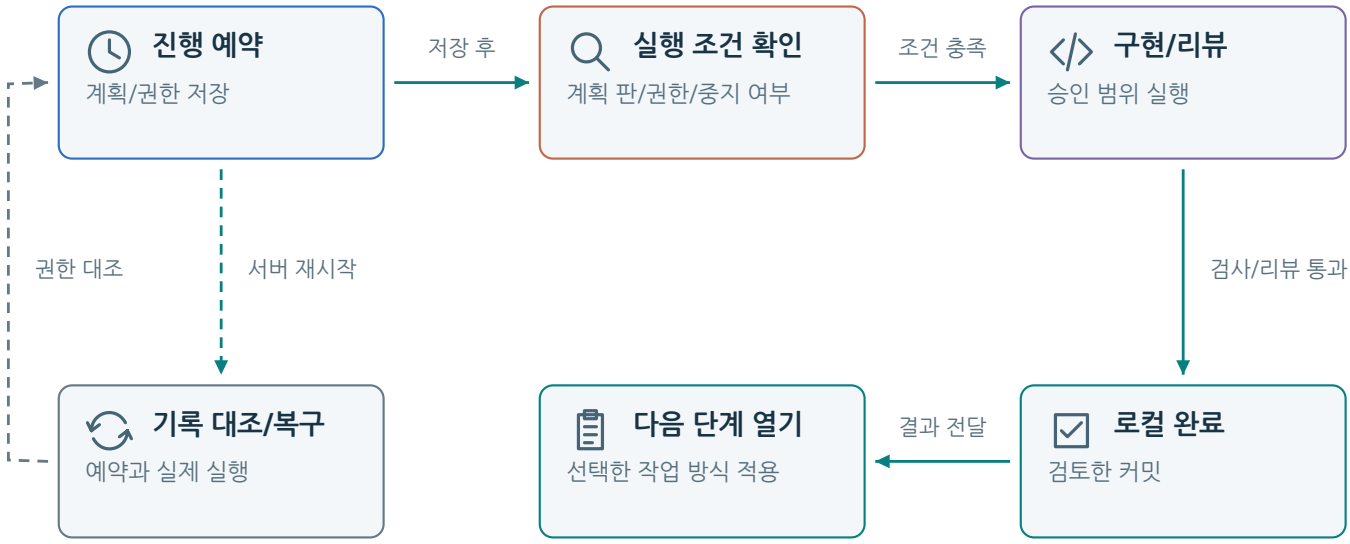
배정을 새로 만들면 기존 신원과 승인 기록이 어긋날 수 있어 조회 규칙을 수정했다. 하위 작업에 명시한 배정은 우선한다. 공개 API에서 두 단계 상속, 서버 재시작 뒤 연결 유지, 하위 재배정 뒤 이전 신원의 거부를 확인했다.

docs/mediation/policy.md / src/server/mediationAutonomy.ts

서버가 맡는 연속 실행

역할 응답으로 다음 턴을 정하고, 승인된 자동 진행 범위만 서버에 맡긴다.

계획 합의가 끝나도 별도 근거 심사, 마무리 확인, 양쪽 확인 응답을 기다리면 구현은 시작되지 않았다. 먼저 누락된 심사 호출을 연결했지만, 한 단계를 고칠수록 다른 복구 조건이 드러났다. 결국 모델이 내용의 합의를 다시 확인하는 사슬을 없앴다.



계획 합의 뒤 멈춘 실행 경로

현재 계획 왕복은 작성/검토/개정으로 끝난다. 리뷰어가 현재 판에 동의하면 사용자 승인을 기다린다. 일반 작업에는 이 계획 게이트가 없다. 구현자는 작업을 마치면 리뷰를 받고, 리뷰어의 수정 요청은 같은 구현 세션으로 돌아간다.

승인된 계획과 재개 위치의 보존

자동 진행을 맡길 때는 계획 판과 허용한 전달 범위를 저장한다. 서버 재시작은 기록을 복구할 뿐, 취소된 권한을 되살리지 않는다. 새 계획은 다시 승인받고, 다음 일반 작업은 열어 둔 뒤 구현 시작을 기다린다.

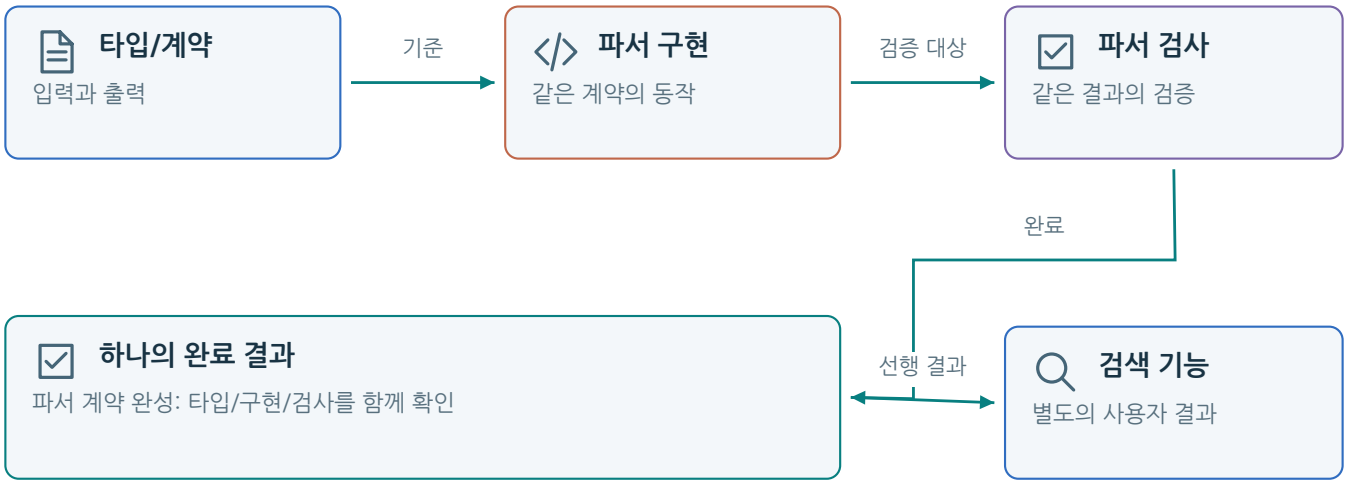
사용자 중지와 오래된 응답도 구분한다. 알림이 늦게 도착해도 먼저 현재 상태를 조회한다. 이미 끝난 실행을 다시 시작하지 않으며, 수신 확인을 작업 완료나 재개 승인으로 해석하지 않는다.

src/server/engine/continuation.ts / src/server/continuationStore.ts / src/shared/workflowLifecycle.ts

완료 결과를 기준으로 한 단계 분리

같은 결과를 만드는 설계/구현/검사는 한 단계 안의 체크리스트로 구성.

설계/구현/테스트를 각각 별도 단계로 만들거나 파일별로 단계를 나누면 같은 내용을 여러 번 설명하고 검토하게 된다. 개편에서는 한 결과를 완성하는 작업을 단계 안의 체크리스트로 묶는다. “이 단계를 끝내면 무엇을 확인할 수 있는가?”를 먼저 적는다.



단계 분리의 판단 기준

분리는 앞 단계의 결과가 있어야 판단할 수 있는 경우, 독립적인 검증이 필요한 경우, 실패 시 따로 되돌릴 필요가 있는 경우로 설명한다. 단계 수나 코드 줄 수를 일률적으로 제한하지 않는다.

현재 단계의 완료 조건

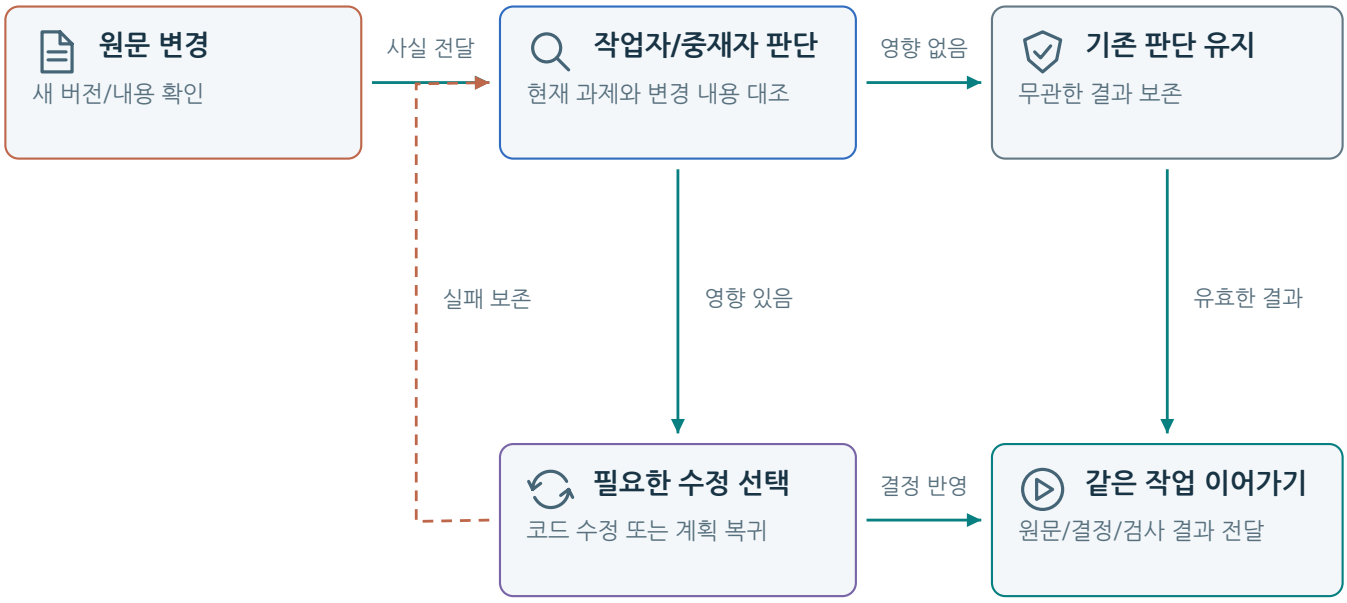
현재 단계에는 완료 조건, 검사 방법, 예산, 해결해야 할 질문이 필요하다. 미래 단계는 개요로 남길 수 있다. 기본적으로 한 단계를 실행하며, 외부 정보나 사용자 결정을 기다릴 때만 의존하지 않는 준비된 단계로 진행할 수 있다. 예산 소진을 기다림으로 바꿔 다른 실행을 시작하지 않는다.

src/shared/workGroups.ts / src/server/workGroups.ts

근거 변경과 판단 보존

원문 변경은 새 사실로 전달하고, 기존 계획/세션/작업 트리는 보존한다.

원문이 바뀔 때마다 계획 승인과 세션을 무효화하면, 구현한 코드가 있어도 다시 계획으로 돌아갔다. 영향 심사를 따로 붙여 해결하려 했지만 심사 실패를 복구하는 경로까지 필요해졌다. 현재는 바뀐 사실을 전달하고, 영향 판단은 작업자와 중재자가 맡는다.



전체 초기화에서 사실 전달로

새 원문, 선택 변경, 조회 실패를 사건으로 남긴다. 각 세션에는 아직 받지 않은 사실을 다음 턴에 보낸다. 시스템이 만든 안내는 사용자 발언으로 저장하지 않는다. 자료 변경이 곧 사용자의 재계획 지시가 되지 않게 한 것이다.

재사용의 범위와 계획 복귀

계획을 바꿔야 하면 중재자가 이유를 적어 같은 플래너에게 돌려보낸다. 검토자 세션은 계획 판과 별도로 이어 쓴다. 계획 방식과 일반 작업 사이를 전환해도 작업 트리, 사건, 사용량과 세션을 초기화하지 않는다.

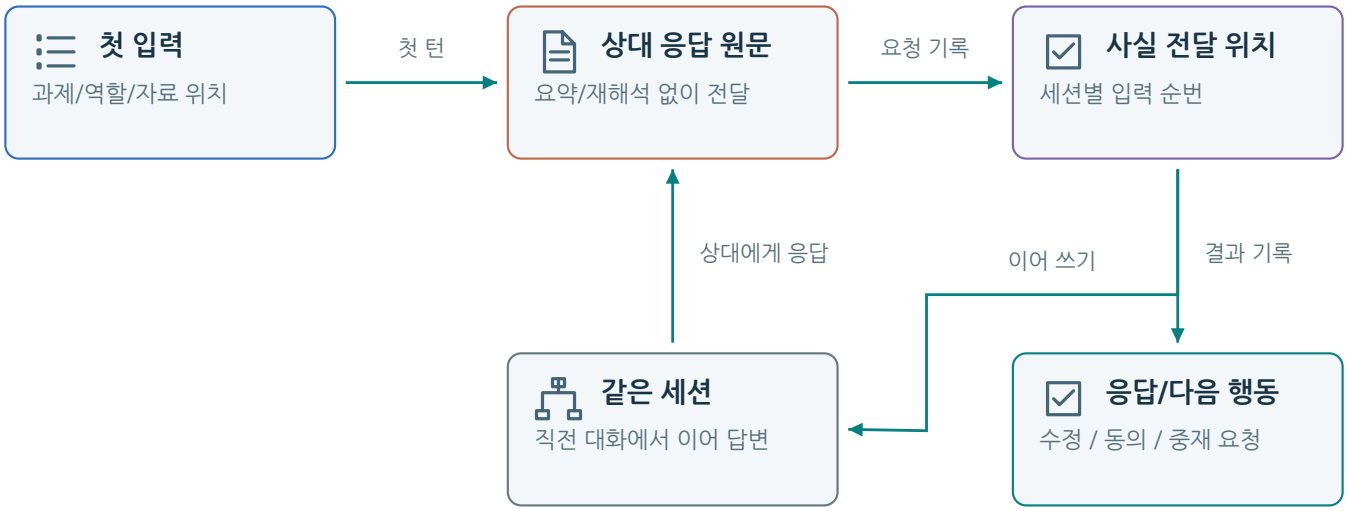
원문 변경이 없다는 사실만으로 모든 결과를 재사용하지는 않는다. 코드/도구/검사 입력을 함께 대조한다. 기계적 초기화를 없앤 만큼, 중재자는 무엇을 다시 확인하고 무엇을 유지했는지 결정에 남겨야 한다.

src/shared/turnContract.ts / src/shared/prompts.ts / src/server/workflow.ts

원문 전달과 대화의 연속성

첫 턴에는 과제와 지침을, 이어지는 턴에는 상대 응답과 새 사실을 전달한다.

옛 계획 제어에서는 요청서와 타임라인을 서버가 조각으로 나눠 읽게 했다. 한 계획 개정은 2,559초가 걸렸고, 64,434바이트 요청서를 처음부터 다시 읽은 기록이 남았다. 분할 크기와 재사용 규칙을 고쳐도 대화를 매번 다시 구성하는 부담은 남았다.



조각 크기를 고치던 시행착오

공통 계약을 따로 떼고 읽은 구간을 재사용하자 일부 재전송은 줄었다. 그러나 읽기 요청, 인용 신원, 완료 여부, 문맥 재구성을 관리하는 규칙이 계속 늘었다. 최종 개편에서는 이 계획 전용 제어와 패커를 삭제했다.

응답 원문을 같은 세션으로 전달

첫 턴에 과제/지침/근거 위치를 보낸다. 다음 턴에는 상대의 응답 원문과 새 사실을 전달한다. 계획 폴더의 현재 판과 변경분은 파일로 남긴다. 여러 Markdown 파일을 지원하되 누적 계획 전체를 매 턴 다시 보내는 방식은 피했다.

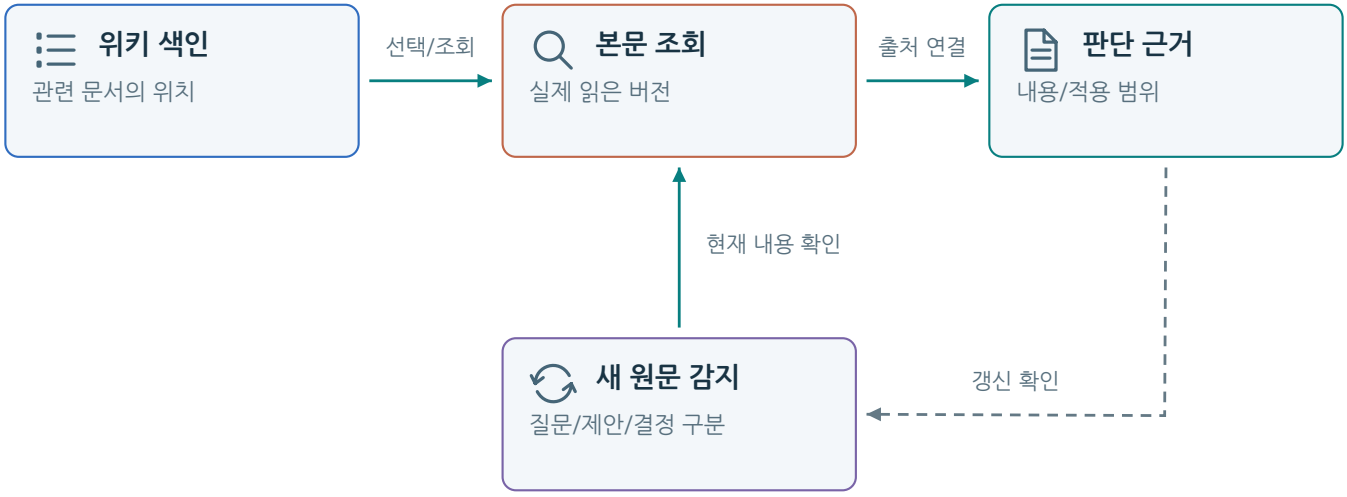
문맥 압축은 Claude/Codex 세션의 기능에 맡긴다. 세션 교체나 역할 인계에서는 사실을 다시 받을 수 있어 중복 전달이 전혀 없다고 주장하지 않는다. 이전 운영 시간과의 같은 조건 비교도 아직 없다.

src/shared/prompts.ts / src/server/engine/planning.ts / src/server/engine/delivery.ts

위키 검색과 실제 근거의 구분

색인에서 찾은 문서와 실제로 읽고 판단에 사용한 원문을 구분.

위키 색인에는 문서의 위치와 관련 주제가 있다. 처음에는 관련 문서 일부만 선택하고, 더 필요한 자료를 질의해 읽는다. 모든 위키를 프롬프트에 붙이거나 색인에 나타난 모든 문서를 판단 근거로 등록하지 않는다.



실제 읽은 문서의 기록

실제로 조회한 문서의 경로/버전/내용을 기록한다. 읽지 않은 문서가 바뀌었다는 이유로 현재 작업을 모두 무효화하지 않는다. 반대로 읽은 원문이 바뀌면 관련 판단을 다시 확인해야 한다.

새 원문의 해석과 검색 비용

위키 본문의 해시와 외부 원문의 해시는 다른 정보를 가리킨다. 외부 자료가 갱신되면 “확인이 필요하다”는 상태를 남긴다. 새 내용이 질문인지 결정인지, 어느 플랫폼에 적용되는지, 기존 근거와 충돌하는지 판단한 뒤 결론을 고쳐야 한다.

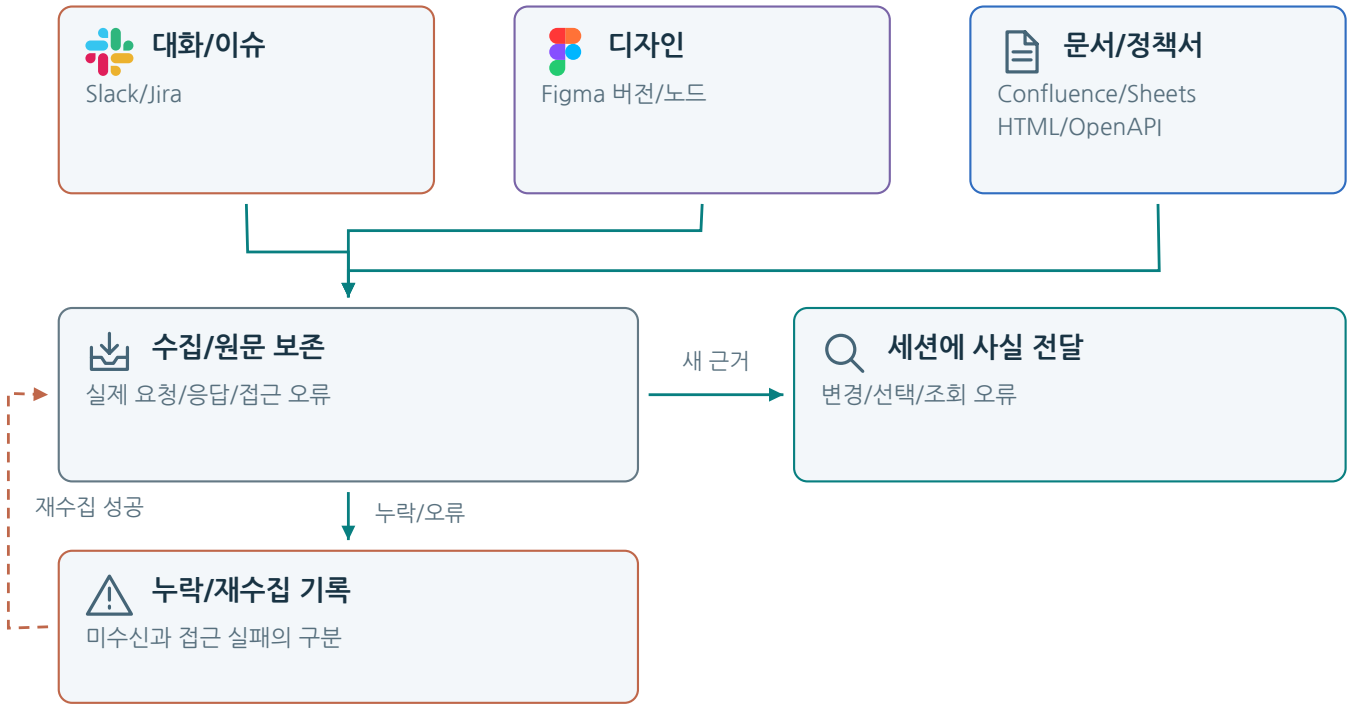
검색 정확도가 좋아졌다고 토큰이 줄었다고 말할 수는 없다. 더 관련 있는 문서를 많이 선택하면 전달량이 늘 수도 있다. 위키 수정 제안도 경로/상태/내용 해시와 역할 권한을 확인한 뒤 반영하며, 폴더 전체를 임의로 고칠 권한을 주지 않는다.

docs/memory-retrieval.md / src/server/wikiEvidence.ts

외부 자료 수집과 변경 전달

자동 수집은 관측 사실을 남기고, 작업 세션이 변경의 영향을 판단한다.

Slack/Jira/Figma/Confluence/Google Sheets와 일반 HTTPS 문서를 등록한다. REST 수집기는 기본 15분 간격으로 재확인하고, 시작할 때 밀린 조회도 처리한다. 특히 요구사항이 자주 바뀌는 티켓모드에서 최신 자료를 작업 세션에 전달하는 데 사용한다. 닫힌 작업에서만 쓰는 출처는 정기 조회에서 제외한다.



수집과 모델 전달의 분리

수집 자체는 모델을 호출하지 않는다. 일반 문서는 HTML 본문과 OpenAPI 명세를 읽으며, 원문/선택 변경은 다음 작업 턴에 전달한다. 작업자와 중재자가 반영 범위를 판단한 뒤 구현 → 검증으로 이어 가므로, 자료를 일일이 확인하고 옮기는 수고가 줄었다. Figma 노드와 화면의 일치는 구현자가 별도로 확인한다.

재확인 기한과 접근 실패의 구분

정상 수집한 원문이 재확인 기한을 넘겼다는 이유로 읽기 대상에서 빠진 적이 있었다. 저장된 내용의 사용과 최신 확인 필요 여부를 나눴다. 접근 오류와 누락은 그대로 기록한다.

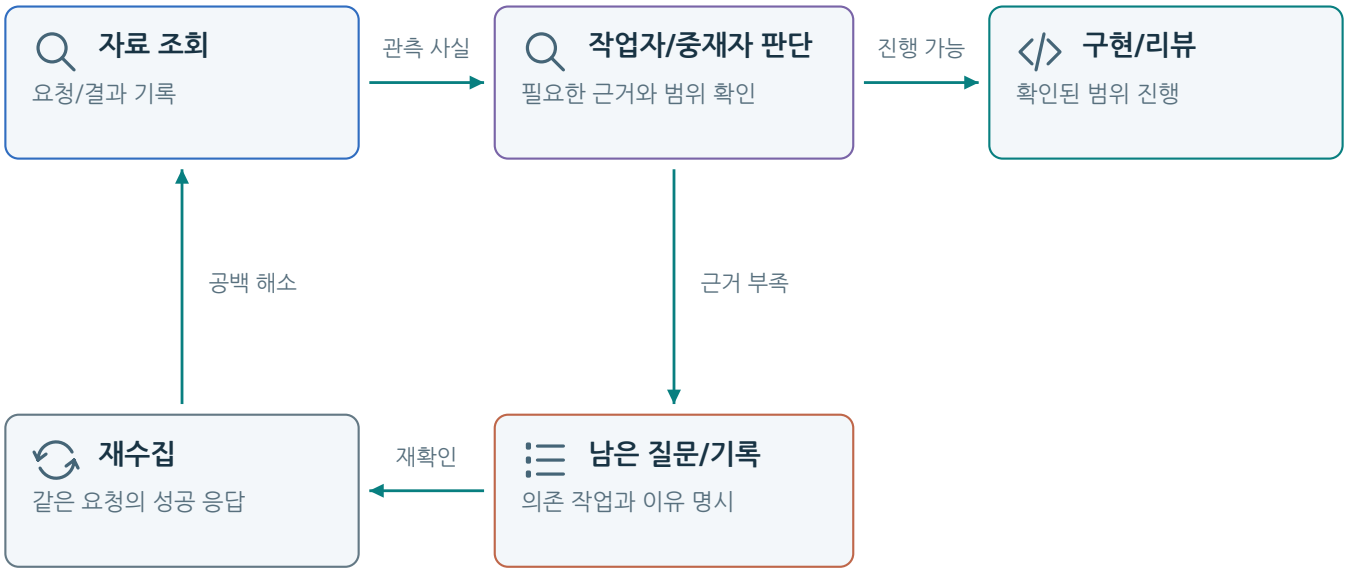
선택에서 뺀 자료도 제거 사실을 전달한다. 같은 실패의 반복 알림은 줄이되, 실제 읽기에 성공하기 전에는 수집 성공으로 바꾸지 않는다. 변경이 계획에 미치는 영향은 작업 세션이 판단한다.

src/shared/externalEvidence.ts / src/server/evidence/discovery.ts / src/server/evidence/service.ts

자료 누락과 후속 작업

부족한 자료에 의존하는 부분만 후속 목록으로 넘기고, 확보한 근거의 범위에서 진행.

근거가 부족한 항목과 진행할 수 있는 항목을 나누려 했지만, 엔진이 공백을 쟁점으로 주입하고 별도 처분까지 요구하면서 판단이 겹쳤다. 구현자/리뷰어의 대화가 할 일을 정하도록 바꾼 뒤에는, 옛 자동 이연 정책도 함께 제거해야 했다.



한 자료의 실패가 전체 중지로 번진 사례

옛 재개 턴에서는 미수신 요청 36개 중 한 Figma 요청이 제한에 걸려 전체 실행이 멈췄다. 먼저 나머지 요청을 보존하고 가능한 일을 이어 가도록 고쳤다. 이후에는 엔진이 이연 쟁점을 합성하는 방식 자체가 필요한지 다시 검토했다.

관측 상태와 작업 처분의 분리

현재 봉투는 역할, 응답 원문, 다음 행동을 담는다. 엔진은 원문 공백을 별도 쟁점으로 만들지 않는다. 구현자나 리뷰어가 판단하기 어려우면 이유를 적어 중재를 요청하고, 중재자는 승인 범위에서 진행/보류를 정한다.

새 봉투가 표현할 수 없는 옛 To-do 필드를 요구하는 안내도 삭제했다. 작업 묶음의 과거 동결 결과에 남은 보류 기록은 출처와 원문을 함께 전달한다. 이는 이전 작업의 참고 기록이며, 새 지시나 자동으로 해결된 항목이 아니다.

src/shared/turnContract.ts / src/server/workGroups.ts / src/server/evidence/store.ts

계획/구현/리뷰의 세션 연결

호환되는 대화와 유효한 읽음 기록을 이어 사용하고, 검토자의 판단 맥락은 별도로 유지.

계획과 구현의 연속성을 살리려면 세션 ID만 보존해서는 부족했다. 이어지는 턴마다 전체 지침을 다시 붙이거나, 계획 변경으로 리뷰어 결속을 지우면 대화는 반복됐다. 현재는 역할별 첫 입력과 이어지는 응답을 구분한다.



대화를 이어 쓰는 조건

계획과 구현의 배정이 호환되고 연속성 정책이 켜져 있으면 계획 세션을 구현에 넘긴다. 코드 리뷰어도 같은 작업 범위에서 이어 쓴다. 계획 판을 바꾸거나 작업 방식을 전환했다는 이유만으로 리뷰어를 새로 만들지 않는다.

세션이 사라지면 실패를 남긴다. 다른 ID를 자동 채택하던 짝 복구 예외는 삭제했다. 새 세션이 필요하면 중재자가 역할과 사유를 명시해 교체한다. 이력은 남기며 작업 전체를 초기화하지 않는다.

검토자의 독립성과 재사용 범위

검토자는 고정된 계획과 코드 변경을 확인한다. 수정 후에는 같은 리뷰 대화에서 지적과 변경분을 확인한다. 형식 위반 교정도 같은 세션에서 한 번 수행하고, 여전히 유효하지 않으면 원문을 보존한 채 실패로 남긴다.

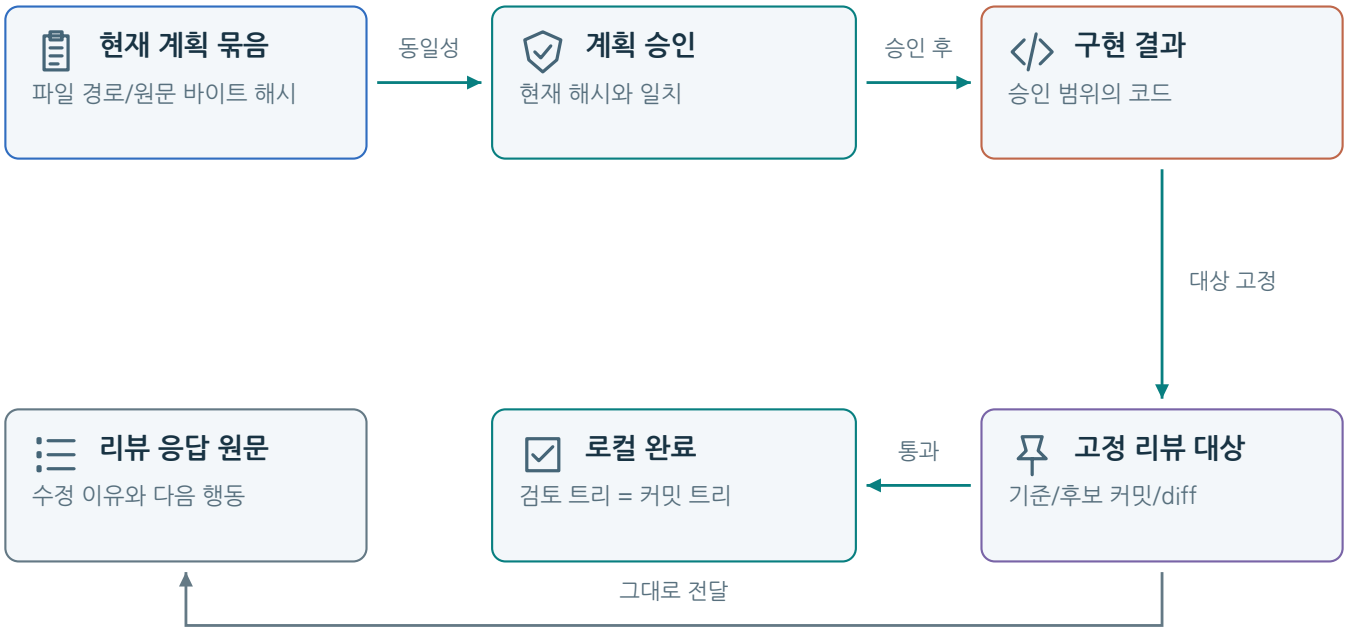
기존 세션을 이어 쓰는 것과 모든 입력의 중복을 없애는 것은 다르다. 인계 시 사실을 다시 받는 경우와 운영 비용은 별도로 확인해야 한다.

src/server/engine/core.ts / src/server/engine/turnExecutor.ts / src/server/planningStore.ts

승인 계획과 검토 코드의 동일성

계획 내용, 리뷰 대상, 실제 결과를 각각 고정하고 같은 대상인지 대조.

계획 작업은 여러 Markdown 파일을 한 묶음으로 확정한다. 상대 경로와 파일 바이트를 정해진 순서로 직렬화하고 SHA-256을 계산한다. 승인한 판과 현재 판이 같아야 구현한다. 일반 작업은 계획 해시를 구현 조건으로 요구하지 않는다.



계획 동일성과 고정된 검토 대상

계획의 버전과 코드의 검토 대상을 구분한다. 코드 리뷰는 HEAD와 변경 내용을 고정하고, 전달 전에도 같은 변경인지 확인한다. 검사를 기다리는 동안 외부 커밋이 끼어들면 리뷰가 다른 코드를 승인하지 못하게 한다.

줄 패치 최적화에서 파일 묶음으로

옛 줄 패치 형식은 한 사례의 수정 데이터를 48,588바이트에서 38,324바이트로 줄였다. 그러나 줄 범위/기준 해시/적용 실패를 관리하는 코드가 필요했다. 현재 플래너는 계획 파일을 직접 고치고, 엔진은 스냅샷과 diff를 저장한다. 옛 패치 적용기는 삭제했다.

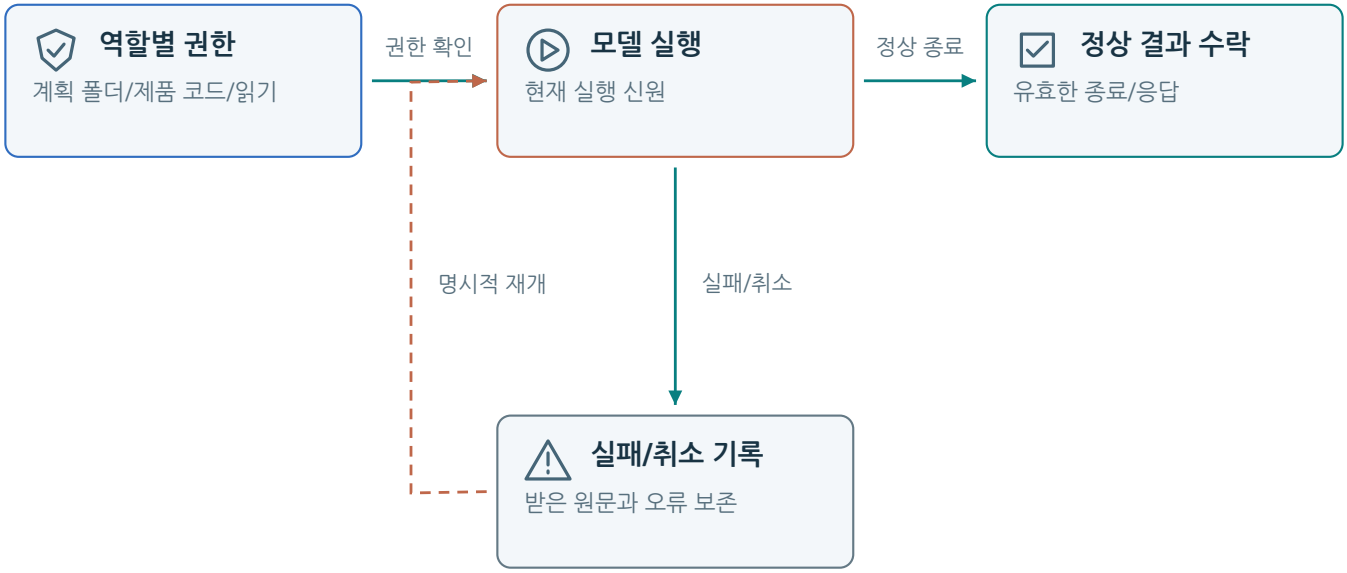
전환 뒤에는 해시의 의미가 어긋난 버그도 드러났다. 묶음의 원문 바이트로 만든 버전을 예전 정규화 함수와 비교해, 바뀌지 않은 계획을 찾지 못했다. 저장과 조회가 같은 바이트 계약을 쓰도록 맞췄다.

src/server/planBundle.ts / src/server/engine/core.ts / src/server/engine/delivery.ts

역할별 권한과 실행 종료

실행 권한/프로세스 종료/결과 형식을 모두 확인한 뒤 결과 수락.

계획 작성자는 지정된 계획 폴더를 쓰고, 검토자는 읽으며, 구현자는 허용된 제품 코드를 바꾼다. 보호할 도구 트리와 역할 권한은 유지한다. 확인을 위한 모델 턴을 없애도 실제 쓰기/실행 경계까지 없애지는 않는다.



응답 수신과 상태 채택의 분리

모델이 결과를 돌려준 뒤 도구 수집 검사나 래퍼의 후처리가 실패할 수 있다. 최종 반환부에서만 결과를 저장하자 이미 받은 원문도 사라졌다. 수신 시점에 원문/세션/역할/요청 당시 입력 순번을 붙잡고, 상태 채택은 이후에 검사하도록 나눴다.

실제 Claude/Codex 어댑터 안에도 후처리 검사가 남아 있었다. 래퍼만 고친 첫 수리로는 충분하지 않아 어댑터의 수신 경계까지 추적했다. 보존했다는 사실을 정상 완료나 재호출 허용으로 해석하지 않는다.

취소한 실행의 응답 차단

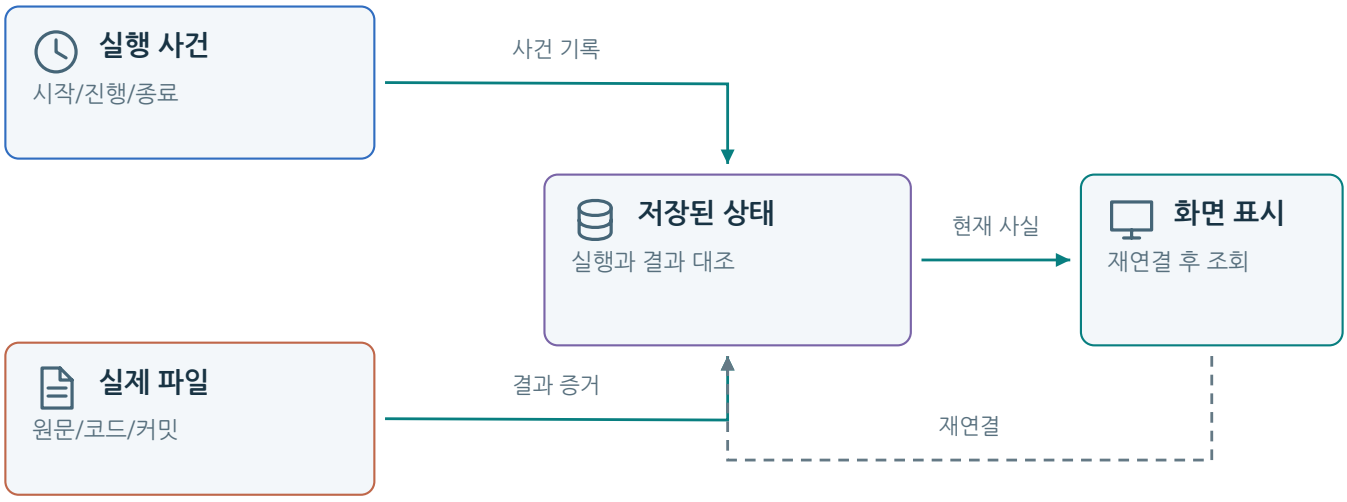
취소/재계획/재시작 뒤에는 실행 신원을 대조한다. 늦은 응답은 최신 상태를 덮어쓰지 못한다. 프로세스를 정리할 때도 PID와 시작 신원을 확인한다. 받은 결과의 보존과 오래된 결과의 채택 거부를 함께 지킨다.

src/server/adapters/claude.ts / src/server/adapters/codex.ts / src/server/engine/core.ts

진행 표시와 실제 작업의 구분

실행 시간/문장 증가/완료 선언을 서로 다른 증거로 확인.

계획/리뷰/구현 결과를 파일로 남기면 대화가 길어져도 필요한 원문을 다시 읽을 수 있다. 다만 별칭 파일이 다른 내용으로 교체될 수 있으므로 내용 해시로 고정한 자료를 기준으로 삼는다. DB의 참조와 실제 파일을 읽을 때 해시를 대조한다.



저장된 상태와 재연결

화면은 서버의 실행 기록과 이벤트를 받아 진행 상태를 보여 준다. 실행 시간이나 누적 문장이 늘어난 것만으로 진척을 판단하지 않는다. 현재 턴의 결과/실제 파일/읽은 자료를 대조하며, 재연결 뒤에도 저장된 상태를 확인한다.

검증 근거별 확인 범위

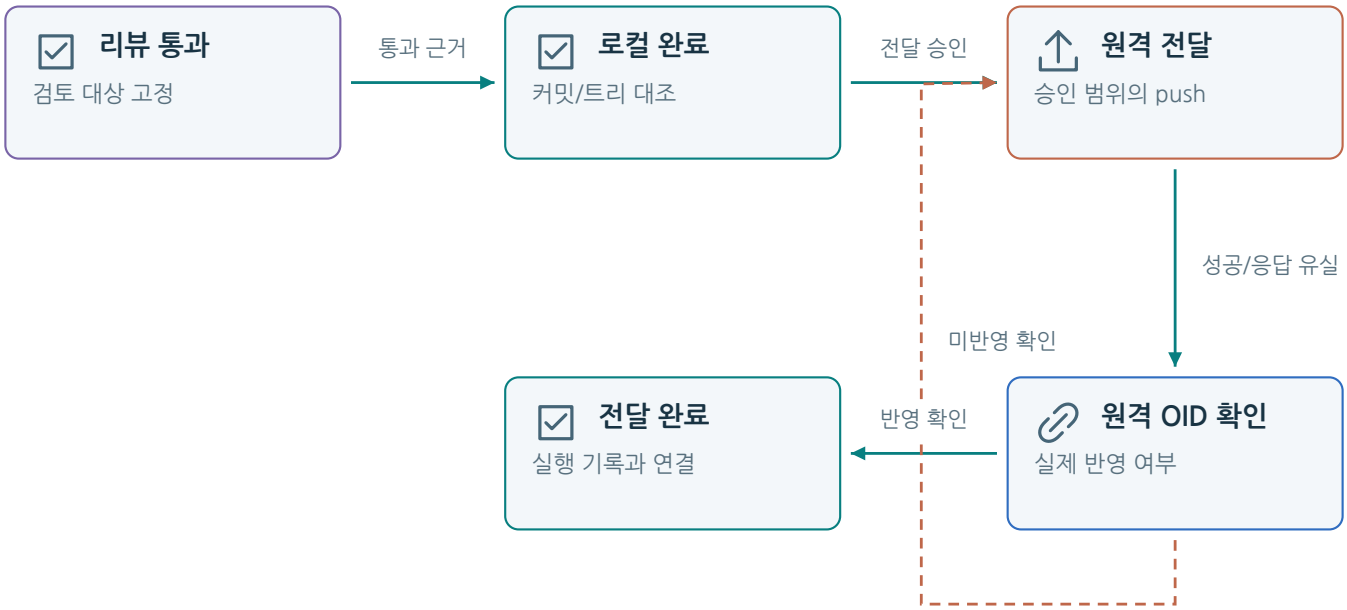
승인한 경로와 변경 규칙을 벗어났는지 확인할 수 있다. Swift 주석/문자열을 구분해야 하는 검사에서는 단순 문자열 치환으로 판단하면 오탐과 누락이 생긴다. 범위 규칙을 적용할 때도 해당 언어의 표현을 고려해야 한다.

서로 다른 증거를 하나의 성공 표시로 합치지 않는 것이 이 설계의 중요한 제한이다.

로컬 완료와 원격 전달

검토한 결과를 로컬 커밋에 연결하고, 원격 반영은 별도 OID로 확인.

리뷰 승인, 로컬 커밋, 원격 전달, 제품 검수 완료는 서로 다른 결과다. 계획 작업은 승인 계획과 코드를 연결하고, 일반 작업도 리뷰한 변경과 전달할 변경을 대조한다. 사용자가 검수 범위를 좁히면 나머지 범위를 완료로 표시하지 않는다.



완료 커밋과 단계 통합

코드를 고정해 리뷰하고, 선택한 파일을 커밋한 뒤 실제 부모/변경 내용/원격 OID를 확인한다. 앞 단계 결과를 합칠 때는 부모 관계도 보존한다. 커밋이나 푸시 성공이 앱 전체의 동작을 검증해 주지는 않는다.

공식 커밋 뒤 추가 검증이 결함을 찾으면 같은 수정/리뷰 세션으로 돌아갈 수 있다. 이때 엔진이 확정한 커밋만 다음 기준으로 인정하고 임의 HEAD는 거부한다. 전달할 커밋의 근거와 다시 연 작업의 근거도 따로 보존한다.

푸시 응답 유실의 복구

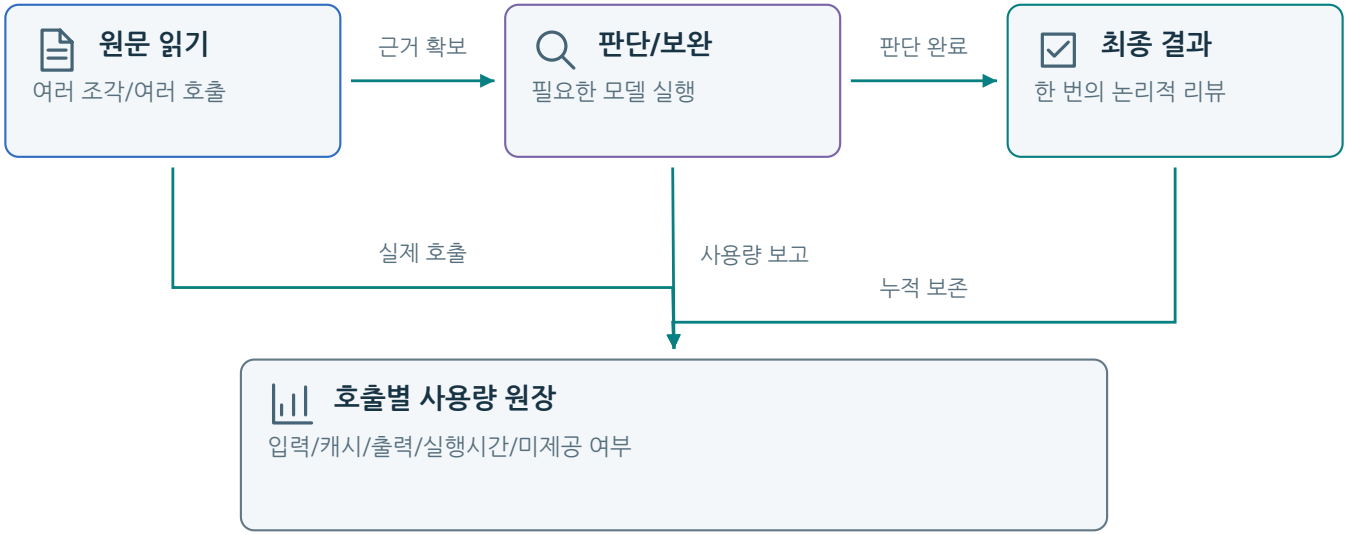
응답을 받지 못해도 원격에는 이미 반영됐을 수 있다. 먼저 원격 커밋과 실행 기록을 대조한다. 이미 전달한 결과를 중복 실행하지 않고, 미반영일 때만 허용된 전달을 다시 시도한다.

src/server/engine/delivery.ts / src/server/evidence/store.ts / src/server/git.ts

사용량 기록과 실행 한도

원문 읽기 횟수, 논리적 리뷰, 실제 모델 실행을 서로 다른 값으로 기록.

대화 왕복 수, 리뷰 회차, 실제 모델 호출 수는 같지 않다. 형식 교정이나 공급자 재시도가 있으면 한 역할 턴에도 여러 호출이 생길 수 있다. 입력/캐시 입력/출력/시간을 호출별로 기록하고, 작업의 한도와 함께 확인한다.



역할별 원문과 실제 실행 횟수

report-usage.py는 운영 DB와 세션 원문을 읽어 역할별 JSON/Markdown 보고서를 만든다. 자동으로 연결할 수 없는 세션은 작업과 역할을 명시하고, 같은 요청의 사용량은 한 번만 합산한다. 역할 턴의 리뷰 횟수와 실제 모델 호출 수는 별도로 남긴다.

누적 사용량과 적용 한도

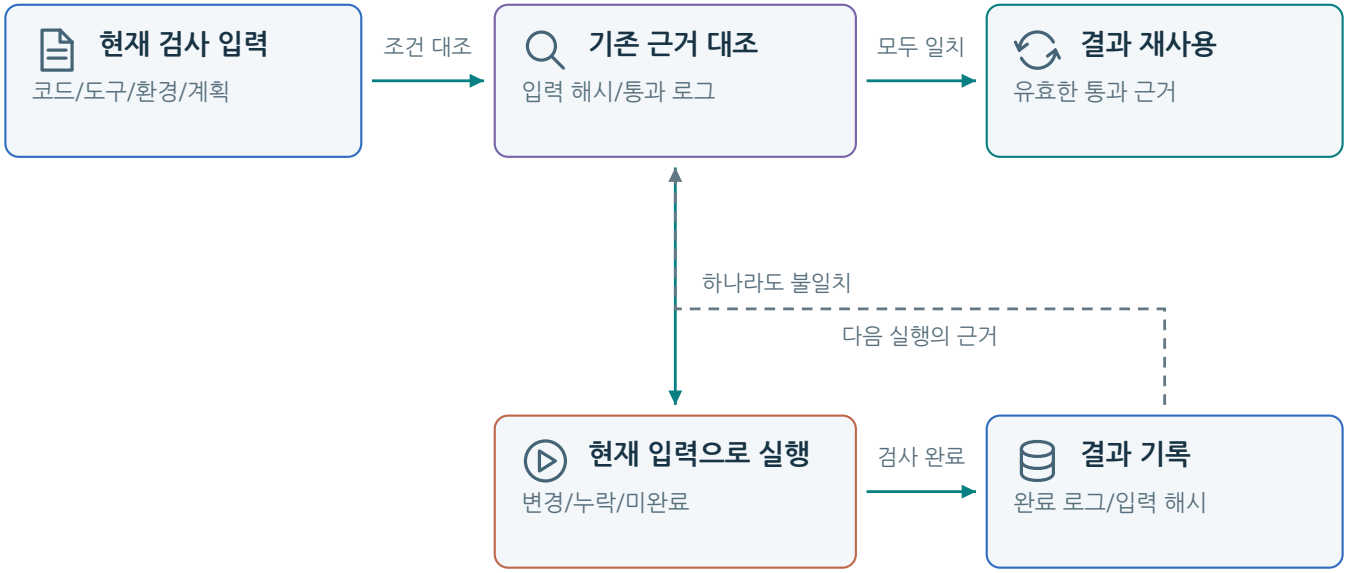
기본 정책은 사용량을 기록하며, 사용자가 정한 예산/횟수와 공급자의 제한은 별도로 적용한다. 새 커밋, 세션 교체, 서버 재시작으로 같은 작업의 사용량을 초기화하지 않는다.

9월 파일럿에서는 당시 예산과 리뷰 회차를 추가했다. 누락된 사용량을 0으로 채우지 않았으며 해당 호출과 응답을 보존했다. 이 비용 기록과 최신 구조의 효과를 같은 측정값으로 합치지 않는다.

검사 결과의 재사용 조건

두 검사 프로필의 입력/도구/환경/계획/로그가 같을 때 통과 결과를 재사용.

Swift 동시성 정책 정적 검사와 변경된 Swift 파일의 구문 검사, 두 프로필을 지원한다. 코드/도구/환경/로그를 대조하고, 계획 작업에서는 승인 계획도 확인한다. 계획 없는 일반 작업도 검사 결과를 저장하고 재사용한다.



같은 검사를 다시 실행하던 비용

작업 단계만 바뀌었다고 같은 검사를 반복하면 실행 비용이 늘어난다. 관련 입력이 같은 통과 결과는 재사용하고, 변경/새 실패/반증이 생기면 해당 검사를 다시 한다. 미추적 Swift 파일도 입력에 포함한다.

항목별 결과와 현재 코드의 대조

계획의 checks 선언은 모든 블록을 읽고 중복 ID를 거부한다. 일부 항목만 실패했는데 전체를 실패로 기록하던 문제도 고쳤다. 상태 전이는 집계로 정하고, 각 항목의 통과/실패/실행 불가를 따로 남긴다.

리뷰 재개에서도 현재 코드로 검사 조건을 확인한다. 검사 중 HEAD가 바뀌면 모델을 부르기 전에 거부한다. 정적 검사 통과는 앱 빌드/화면/API 성공의 근거로 확장하지 않는다.

src/server/planCheckGate.ts / src/shared/planChecks.ts / src/server/verifications.ts

9월 파일럿의 실행 경로

첫 실패는 보존하고, 엔진 중간 수정 없이 새 파일럿에서 다섯 단계와 원격 전달까지 확인.

첫 파일럿은 예산 처리와 계획 확정 경로의 결함으로 중단됐다. 중단 기록을 유지한 채 관련 계약을 수정하고 검토했다. 그 실행을 이어 붙여 성공으로 만들지 않고 새 파일럿에서 전체 흐름을 다시 확인했다.



개발 네 단계와 최종 통합

두 번째 파일럿은 개발 네 단계와 최종 통합을 실제 Claude/Codex CLI로 수행했다. 진행 중 엔진 파일을 바꾸지 않았고, 비교 대상 243개 파일의 해시가 유지됐다. 마지막에는 모든 단계를 닫고 최종 통합 결과를 격리된 bare 원격에 전달했다.

대상 검사와 실패 검출

최종 대상 프로젝트의 검사 77개가 통과했고, 고의로 필요한 export를 제거한 반대 검사에서는 실패했다. 통과 숫자만으로 판단하지 않고 검사가 실제 결함을 거부하는지도 확인했다.

docs/usage-report.md / 9월 파일럿 당시 관측 결과

파일럿 사용량과 검증 범위

9월 파일럿의 사용량과 10월 개편의 검사 결과는 서로 다른 실행의 증거다.

9월 파일럿은 실제 Claude/Codex CLI를 사용해 다섯 단계를 수행하고 격리된 원격에 결과를 전달했다. 아래 수치는 그 실행에서 수집한 관측값이다. 비교 조건이 같은 이전 실행은 없으므로 이 값을 비용이나 시간의 절감률로 해석하지 않는다.

입력 토큰 22,602,633



캐시 입력 18,134,381
전체 입력에 포함

그 밖의 입력 4,468,252

출력 620,113 토큰 / 실행 48회 / 호출 시간 합계 6,566초

캐시 입력과 실행시간의 해석

캐시 입력은 전체 입력에 포함되며 다시 더하지 않는다. 6,566초는 호출 시간의 합계로 전체 작업의 경과 시간과 다르다. 파일럿은 예산/리뷰 횟수 추가가 있었고, 외부 자료도 저장된 시험용 스냅샷이었다. 완전 무인 실행으로 소개하지 않는다.

최신 엔진 검사와 제품 검수의 구분

10월 9일 흐름 단순화 후보의 실행 기록에서 엔진 검사 1,686개와 호스트 리뷰 검사 275개 통과를 확인했다. 조건부 8개는 건너뛰었고 TypeScript/웹 빌드/공식 리뷰를 확인했다. 코드와 검사 입력이 같은 통과 결과는 재사용했다.

대상 iOS 앱은 매물 등록 2단계의 화면/입력/로컬 검증/임시저장 복원을 검수했다. 관련 검사 980개와 디자인 시스템 179개가 통과했고, 같은 CR 리뷰어가 추가 지적 없이 승인했다. 다른 단계와 매물 전송 API 성공은 이 승인에 포함하지 않는다.

docs/usage-report.md / 9월 파일럿과 10월 개편/제품 검수는 별도 실행

실제 iOS 앱의 Swift 6 전환

2026.08.30 - 09.23, 약 4주에 걸친 전환. 10월 개편 전 엔진으로 수행한 운영 사례.

운영 중인 iOS 앱을 Swift 6로 전환하면서 Consensus Room을 사용했다. 공통 모듈부터 기능 모듈/앱/테스트까지 의존 순서로 작업을 나누고, 각 단계의 계획/구현/검토 기록을 보존했다.



각 단계에서 계획 합의 → 승인 → 구현 → 교차 검토 → 검증

범위와 판단은 사람이 정하고, 두 모델의 응답/지적/수정 결과를 같은 작업에 보존

직접 맡은 판단과 모델별 역할

작업 범위와 설계 판단, 계획 승인은 직접 맡았다. Claude의 구현을 Codex가 교차 검토하고, 지적과 수정 내역을 같은 단계에 남겼다. 예외가 필요한 동시성 처리는 적용 범위와 근거를 기록해 다음 단계에서도 확인했다.

검토가 실제 수정을 만든 사례

공통 모듈의 전환 전후 비교에서 프로젝트 생성이 실패했는데도 기존 Swift 6 프로젝트로 두 빌드를 수행한 문제가 드러났다. 생성 성공과 실제 컴파일 언어 모드를 먼저 확인하도록 절차를 고치고, Swift 5와 Swift 6의 비교 결과를 다시 수집했다.

전환 완료와 재유입 방지

9월 23일 전환 코드를 개발 브랜치에 통합하고, 앱/테스트를 포함한 프로젝트의 Swift 언어 모드 기본값을 6.0으로 올렸다. 전환 후에는 Swift 5 설정이 다시 들어오면 실패하도록 CI 검사를 추가했다. 구현 완료뿐 아니라 후속 변경에서도 전환 결과를 지킬 수 있게 했다.

단계별 계획/구현/검토 기록 / iOS 전환 커밋 / Project.swift / ci/check_swift_version_policy.py

복구 계약의 검증과 남은 과제

기존 작업을 초기화하지 않고 새 흐름으로 옮긴 뒤, 실제 재개와 결과 보존을 확인했다.

새 구조를 만드는 동안 기존 정지 작업도 이어 가야 했다. 기존 계획/미커밋 코드/세션/사용량을 버리고 새 토픽으로 시작하면 보존을 증명할 수 없다. 옛 저장 상태를 먼저 이행하고, 현재 타입으로 기동 복구를 수행하도록 순서를 바꿨다.



복구 이후 실제 진행 확인

옛 상태를 현재 스키마로 먼저 읽으면 파싱이 실패하거나 재개 지점이 사라졌다. 원시 행의 이행을 그 읽기보다 앞으로 옮겼다. 사용자의 중지를 유지하고, 저장된 대기 표식도 보존한다. 재시작만으로 모델 턴을 시작하지 않는다.

새 흐름의 첫 실제 실행에서는 일반 작업의 과제가 첫 입력에서 빠진 결함도 찾았다. 수동 결정문으로 보완할 수 있었지만, 저장된 과제가 자동 전달된다는 계약은 깨져 있었다. 기존 과제 렌더를 재사용해 구현자/리뷰어 첫 입력에 넣었다.

다음 검증 범위와 설계 기준

매물등록 폼의 단계 구성/서버 기반 검증/안내 문구는 개별 작업으로 완료했다. 통합 검증은 작업 트리의 HEAD가 엔진에 기록된 구현 기준과 달라 중단됐고, 그 실패 기록은 유지했다.

별도로 정한 2단계 화면/입력/임시저장 범위는 검수와 리뷰를 마쳤다. 앞선 통합 검증이나 다른 단계/전송 API까지 성공한 것으로 합치지 않았다.

src/server/workflowMigration.ts / src/server/engine/delivery.ts / docs/mediation/policy.md

오류 교정과 원문 보존

실패를 숨기지 않고 받은 결과를 보존하며, 같은 원인을 처리하는 지점을 줄였다.

받은 원문: 먼저 보존

응답/세션/요청 시점의 커서
정상 완료 판정과 분리

실행 경계: 조건 확인

권한/취소/세션/Git HEAD 유지
같은 세션 교정 또는 명시적 재개

요청 교정만으로 풀리지 않았던 문제

옛 계획 제어는 잘못된 자료 이름을 오류 응답으로 돌려주고 읽기 위치를 보존했다. 그러나 이런 보완을 늘려도 계획 패커와 인용 검사를 복구하느라 제품 작업이 멈췄다. 새 계획 왕복은 이 전용 제어를 제거하고 같은 세션이 파일을 읽고 응답하도록 바꿨다.

수신 결과를 잃지 않는 공통 처리

결과를 받은 뒤 계획 파일 읽기, Git 조회, 래퍼 후처리가 실패하는 경로를 따로 추적했다. 수신 결과와 그 요청의 입력 순번을 먼저 보존하고, 정상 채택 뒤에는 임시 보존 슬롯을 비운다. 늦은 결과를 두 번 저장하는 회귀도 검사했다.

남겨야 할 검사와 없애야 할 예외

형식이 틀린 응답은 같은 세션에서 한 번 교정한다. 권한/취소/세션 신원/Git HEAD 검사는 유지한다. 반면 옛 세션 계보에 짝이 있다는 이유로 다른 세션 ID를 받아들이던 예외는 삭제했다. 새 세션은 중재자의 명시적 교체로 연다.

src/server/engine/core.ts / src/server/engine/turnExecutor.ts / src/server/adapters/

자동화의 한계와 설계 기준

지원 범위 안에서 작업을 제대로 끝내고, 실패해도 이해하고 이어 갈 수 있는 구조를 목표로 삼았다.

중재자: 맥락에 따른 판단

지적의 타당성/우선순위/근거 이연
승인된 범위에서 다음 행동 선택

엔진: 실행 조건과 기록

계획/코드 버전/권한/필수 검증 확인
실행 식별/저장된 결과/재개 위치 관리

예외를 늘릴수록 제품 작업은 더 늦어졌다

처음에는 중단마다 복구와 확인 단계를 더하면 안정적으로 진행될 것이라 봤다. 실제로는 계획 읽기, 지적 분류, 근거 심사, 재확인이 서로의 상태에 의존했다. 엔진 수리를 끝내야 앱 작업을 이어 갈 수 있는 상황이 반복됐다.

같은 원인의 판단을 모으고 대체된 옛 경로를 삭제했다. 다만 삭제 중에도 간접 소비처와 공개 계약을 놓쳐 착수 범위를 여러 번 고쳤다. 작게 나누는 것 자체보다, 한 변경이 완결되는 의존 관계를 먼저 확인하는 일이 중요했다.

판단을 위임하고 실행 경계는 유지

현재는 작업자끼리 원문 응답을 이어 주고 중재자가 범위/우선순위를 정한다. 엔진은 권한, 중복 실행, 실행 신원, 결과와 중단 위치를 보존한다. 긴 예외 규칙을 프롬프트로 옮기는 데 그치지 않고, 실제 분기/확인 턴/옛 원장을 제거했다.

완료와 유지 비용으로 개선을 판단

이번에는 기존 작업을 보존한 채 사용자가 정한 2단계까지 완료했다. 모든 상태와 외부 API의 성공을 보장한 것은 아니다. 남은 낮은 등급 문제는 후속으로 두었다. 운영 완료율/개입 시간/토큰 비용의 같은 조건 비교는 앞으로의 측정 과제다.